

Problem 1: CNN Training on LeNet-5

I. Abstract and Motivation

In this section, we investigate the performance of the LeNet-5 Convolutional Neural Network(CNN) architecture across three benchmark datasets: MNIST, Fashion-MNIST, and CIFAR-10. The motivation is to understand CNN architecture and how hyper parameters affect model convergence and performance. Additionally, this section conducts an ablation study utilizing confusion matrices and One-vs-Rest ROC curves to analyze class specific confidence. Finally, this section evaluates the model's robustness by synthesizing Symmetric Label Noise (SLN) on the Fashion-MNIST dataset to observe how deep learning models behave under varying levels of corrupted training labels.

II. Approach and Procedures

II.1 CNN Architecture

1. CNN Components

- **Fully Connected Layer:** Located toward the end of the network, this layer connects every neuron from the previous layer to every neuron in its own layer. Its purpose is to aggregate the high-level features extracted by the convolutional base and use them to make final predictions.
- **Convolutional Layer:** This is the core building block of the network. It applies a set of learnable filters(kernels) across the input data to perform a convolution operation. Its function is feature extraction which is identifying specific visual patterns like edges, textures, or shapes while preserving the spatial hierarchy of the image.
- **Max Pooling Layer:** This layer slides a window over the input feature map and passes on only the maximum value from within that window. It downsamples the spatial dimensions, which reduces the computational load and number of parameters. Max Pooling Layer also provides translation invariance, meaning the network can recognize a feature even if its position shifts slightly.
- **Activation Function:** Applied after the convolution, this operation introduces non-linearity into the system. Its function is to enable the network to learn complex, non-linear representations. Without Activation Function, stacking multiple layers would mathematically collapse into a single linear transformation.

- **Softmax Function:** Typically used as the final activation in the output layer for multi-class tasks. Softmax Function converts raw output scores(logits) into a normalized probability distribution, ensuring all outputs fall between 0 and 1 and sum to exactly 1.

2. Over-fitting and Mitigation in CNN

Over-fitting occurs when a model learns the training data too much, memorizing the noise, outliers, and specific details rather than grasping the general patterns. As a result, the model shows high accuracy on the training set but performs poorly on unseen validation or test data because it fails to generalize. A standard technique used in CNNs to avoid over-fitting is Dropout. During training, Dropout randomly deactivates a specified percentage of neurons at each update step. Dropout prevents neurons from co-adapting too heavily to the specific outputs of neighboring neurons, forcing the network to distribute its learning and develop more robust, independent feature detectors.

3. Differences Between ReLU, Leaky ReLU, and ELU

- **ReLU(Rectified Linear Unit):** $f(x) = \max(0, x)$. It outputs the input directly if it is positive, and outputs zero if negative. It is computationally efficient and helps solve the vanishing gradient problem for positive values. However, it suffers from the dying ReLU problem (if a neuron only receives negative inputs, its gradient becomes permanently zero, rendering it inactive).
- **Leaky ReLU:** Defined as $f(x) = \max(\alpha x, x)$, where α is a small constant (ex., 0.01). It fixes the dying ReLU issue by allowing a slight, linear slope when the input is negative. This ensures that a small, non-zero gradient flows through, allowing inactive neurons to potentially recover during training.
- **ELU(Exponential Linear Unit):** Defined as $f(x) = x$ for $x > 0$, and $f(x) = \alpha(e^x - 1)$ for $x \leq 0$. Unlike Leaky ReLU's straight line, ELU curves exponentially for negative values. This smooths the activation function and brings the mean activation closer to zero, which often speeds up learning convergence. The tradeoff is that the exponential calculation makes it computationally more expensive than the other two.

4. Loss Functions

- **L1 Loss (Mean Absolute Error):** L1 Loss function is used for regression tasks where the dataset contains significant outliers (ex. predicting real estate prices or stock market anomalies). It calculates the absolute difference between predicted and actual values. Because it does not square the error, it is robust to outliers. Extreme anomalies will not disproportionately dominate and skew the gradient during training.
- **MSE Loss (Mean Squared Error):** MSE Loss function is used for standard regression tasks without extreme outliers (ex. predicting age, temperature or depth estimation). It calculates the squared difference between predictions and targets. This heavily penalizes large errors over small ones, driving the model to predict the mean of

the target distribution. It also provides a smooth gradient landscape that aids stable convergence.

- **BCE Loss (Binary Cross Entropy):** BCE Loss function is used for binary classification tasks(ex. spam detection, medical diagnosis of a specific tumor or passing/failing an inspection). It measures the information theoretic difference between two probability distributions. It heavily penalizes highly confident but incorrect predictions, effectively pushing the model’s output probability toward 0 or 1 for the two distinct classes.

II..2 Compare Classification Performance on Different Dataset

The LeNet-5 consists of two convolutional layers(kernel size 5×5 , stride 1) followed by max pooling, and three fully connected layers. I employed the ReLU activation function for hidden layers and Softmax for the output probabilities. To find the optimal configuration, I first tested three representative hyper parameter settings on the MNIST dataset using five independent runs per setting to ensure statistical significance:

1. **Setting 1:** SGD Optimizer (Learning Rate = 0.01, Weight Decay = 0, Momentum = 0.9)
2. **Setting 2:** Adam Optimizer (Learning Rate = 0.001, Weight Decay = 0)
3. **Setting 3:** Adam Optimizer (Learning Rate = 0.001, Weight Decay = $1e-4$)

After observing convergence behaviors, I also selected the optimal setting and trained LeNet-5 on Fashion-MNIST and CIFAR-10 to achieve the target accuracies ($\geq 90\%$ and $\geq 65\%$, respectively).

II..3 Evaluation and Ablation Study

Using the optimal models trained in the previous step, I performed a thorough ablation study. For each dataset, I computed the normalized confusion matrix on the test set and extracted the top three most confused class pairs. Furthermore, I evaluated the CIFAR-10 model using the One-vs-Rest Receiver Operating Characteristic curve. Because accuracy only measures discrete top 1 predictions, the ROC curve and the weighted area under the curve (AUC) were calculated using the continuous Softmax confidence scores to provide a deeper understanding of the classifier’s ranking capabilities.

II..4 Classification with Noisy Data

For a given noise level ϵ , a uniform random variable was drawn for each sample. With probability ϵ , the true label i was discarded and replaced by a uniformly chosen incorrect label $j \neq i$. This yields a theoretical transition matrix where the diagonal elements are $1 - \epsilon$ and the off-diagonal elements are $\frac{\epsilon}{9}$. I trained the network from scratch under $\epsilon \in \{0\%, 20\%, 40\%, 60\%, 80\%\}$ to observe performance degradation.

III. Experimental Results

III.1 Compare Classification Performance on Different Dataset

1. Hyper Parameter Search with MNIST:

- **Setting 1 (SGD):** Best: 99.29%, Mean: 99.17%, Std: 0.08%
- **Setting 2 (Adam, no WD):** Best: 99.22%, Mean: 99.11%, Std: 0.08%
- **Setting 3 (Adam, WD=1e-4):** Best: 99.21%, Mean: 99.15%, Std: 0.05%

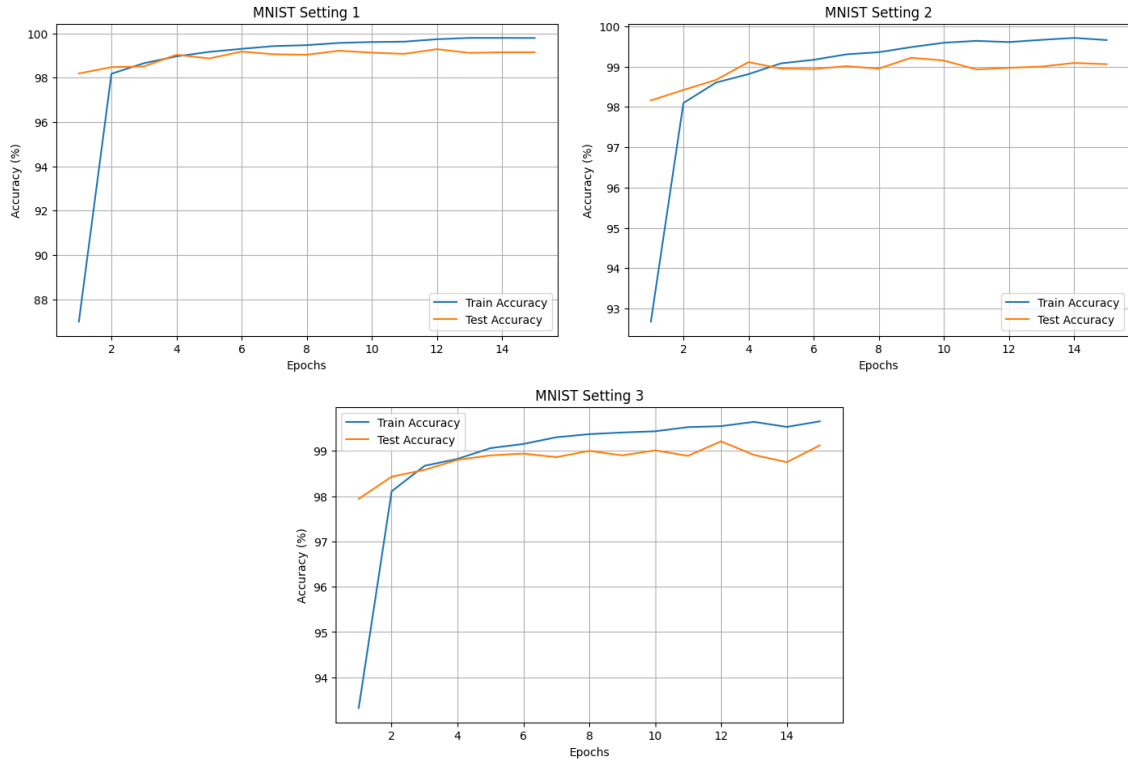


Figure 1: Performance curves for three different hyper parameter settings on MNIST.

2. Optimal Performances: The best testing accuracies achieved across 5 runs for each dataset's optimal setting were:

- **MNIST (Optimal: SGD, LR=0.01, WD=0):** Best: 99.29%, Mean: 99.17%, Std: 0.08%
- **Fashion-MNIST (Optimal: Adam, LR=0.001, WD=0):** Best: 90.56%, Mean: 90.38%, Std: 0.14%
- **CIFAR-10 (Optimal: Adam, LR=0.001, WD=1e-4):** Best: 65.02%, Mean: 63.89%, Std: 0.81%

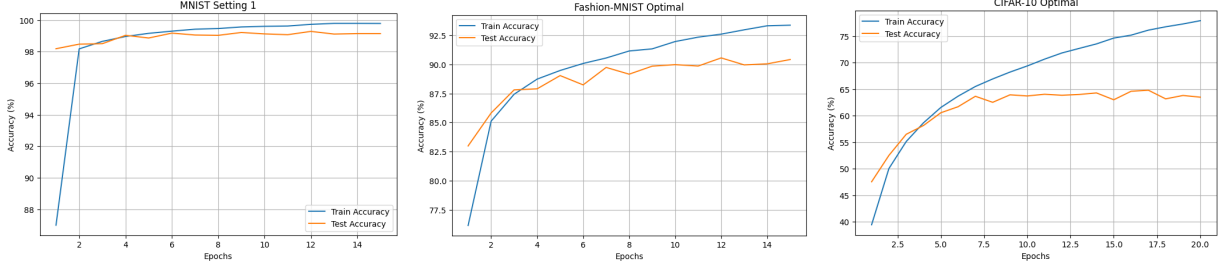


Figure 2: Optimal training and testing accuracy curves for the three datasets.

III.2 Evaluation and Ablation Study

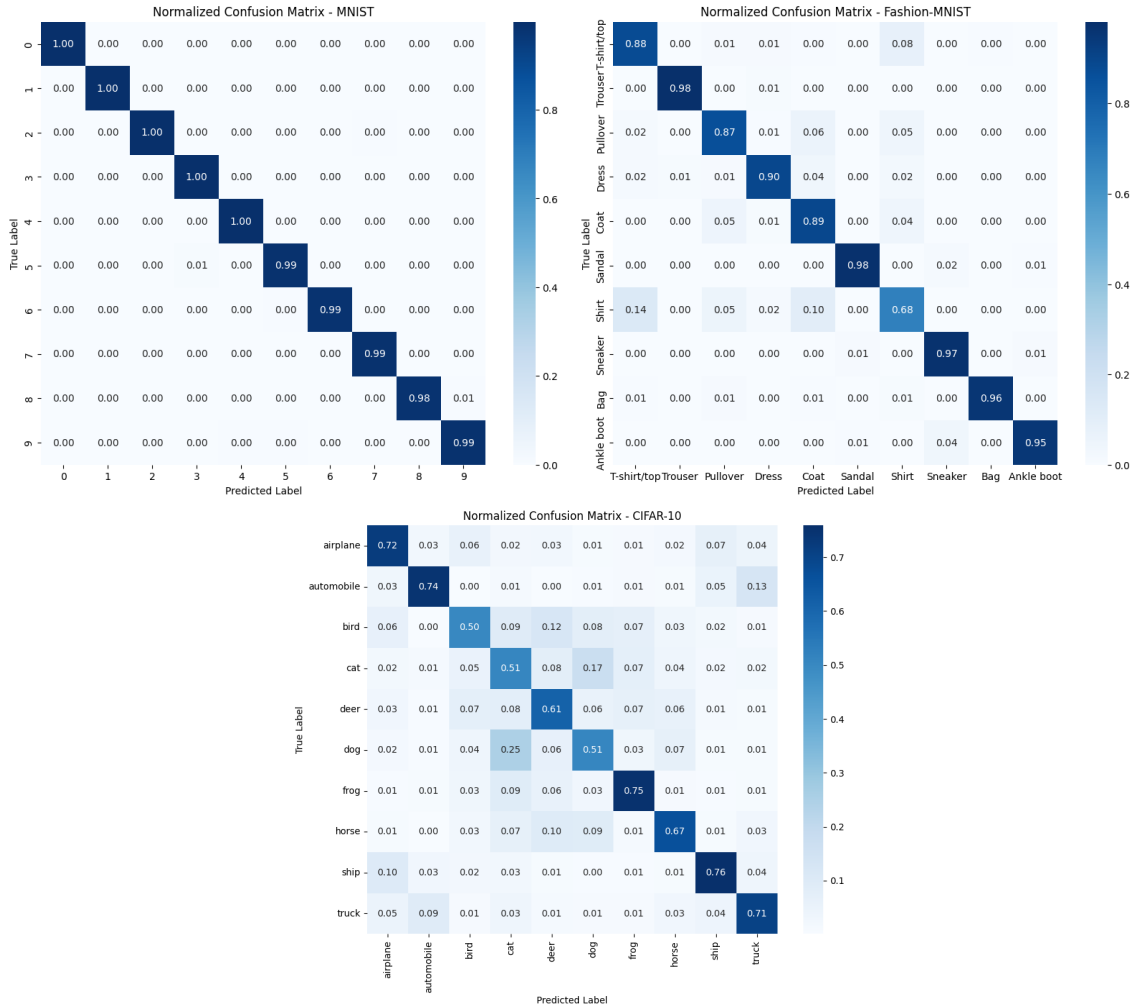


Figure 3: Normalized Confusion Matrices for MNIST, Fashion-MNIST, and CIFAR-10.

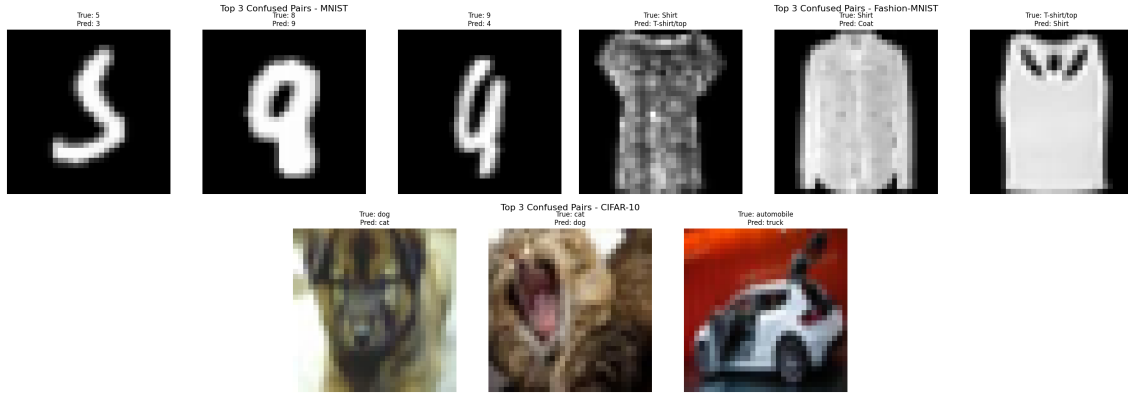


Figure 4: Top 3 confused image pairs for each dataset.

The calculated weighted area under the ROC curve for CIFAR-10 is **0.9386**.

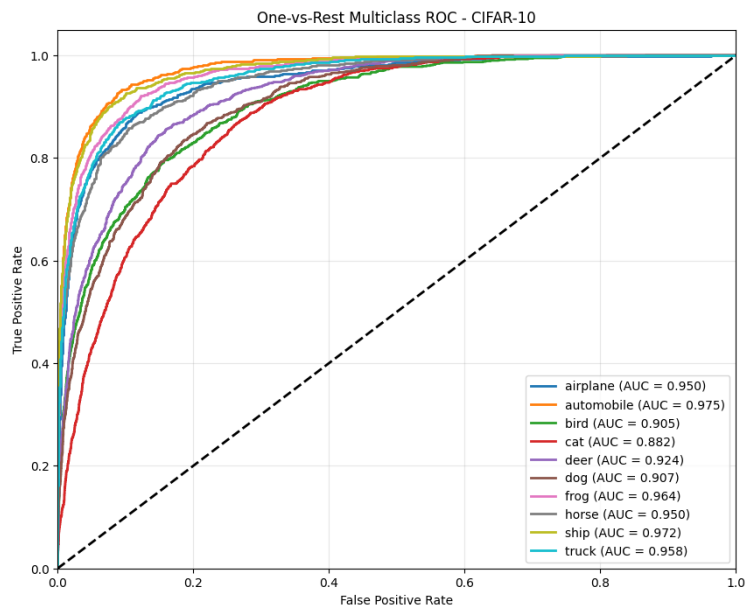


Figure 5: One-vs-Rest Multiclass ROC Curve for CIFAR-10.

III.3 Classification with Noisy Data

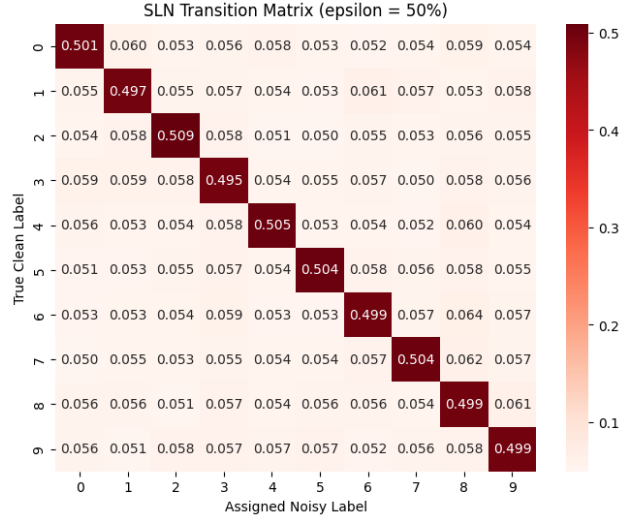


Figure 6: Symmetric Label Noise Matrix at $\epsilon = 50\%$.

Table 1 shows the performance of LeNet-5 on Fashion-MNIST under varying levels of SLN (5 runs each).

Noise Level (ϵ)	Mean Test Accuracy (%)	Standard Deviation (%)
0%	89.84	0.27
20%	89.05	0.13
40%	87.82	0.17
60%	86.15	0.39
80%	78.56	0.90

Table 1: Mean and Std of Testing Accuracy vs. Noise Level

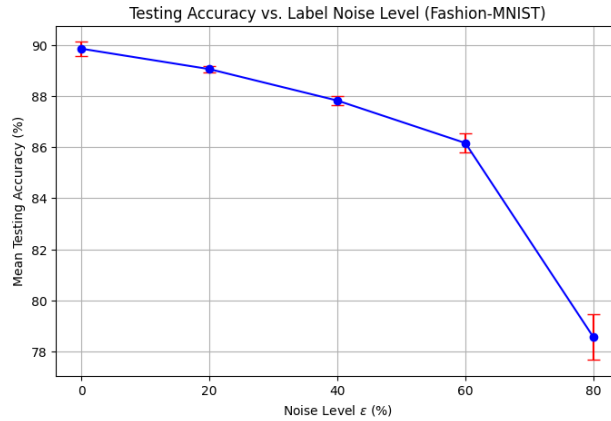


Figure 7: Testing Accuracy Degradation Curve vs. Symmetric Label Noise Level

IV. Discussion

IV..1 Compare Classification Performance on Different Dataset

While SGD converges smoothly, Adam optimizes much faster in early epochs. Adding a slight weight decay in Setting 3 helped shrink the generalization gap between the training and testing curves, reducing overfitting. The significant performance drop from MNIST (99%) to CIFAR-10 (65%) is due to data complexity. MNIST consists of simple, high contrast structural edges. Fashion-MNIST is also grayscale but has higher intra-class variance (ex. distinguishing different types of shirts). CIFAR-10 contains natural color images with complex backgrounds, occlusion, and varying perspectives. LeNet-5 is a shallow network. It lacks the necessary receptive field and parameter capacity to model the highly complex semantic features required for CIFAR-10.

IV..2 Evaluation and Ablation Study

In MNIST, the network confuses structurally similar digits like (3, 5), (9, 8) and (4, 9). In Fashion-MNIST, the top confusions are between Shirt, T-shirt/top, and Coat due to their similar spatial silhouettes. In CIFAR-10, Cat and Dog are highly confused because they share low level features like four legs, fur and similar shapes that simple convolutions cannot separate well at 32×32 resolution.

While the CIFAR-10 top 1 accuracy is only 65%, the weighted AUC is incredibly high at 0.9386. Accuracy only credits the absolute highest probability prediction. The ROC curve proves that even when the network is wrong, it is very well calibrated. It consistently ranks the true class as the 2nd or 3rd most probable option, separating it from the completely unrelated classes.

IV..3 Classification with Noisy Data

The noise degradation curve reveals that deep neural networks possess strong robustness to symmetric noise. From 0% to 40% noise, the mean accuracy drops by only 2%. This occurs because the true label signal remains the most frequent consensus during gradient descent, allowing the network to generalize the underlying patterns and mostly ignore the uniformly distributed noisy gradients. However, once ϵ becomes 80%, the false labels aggressively overpower the true signal, forcing the network to memorize random noise rather than learn generalized features. This results in a sharp drop in accuracy (78.56%) and a significant increase in variance (0.90%) as optimization becomes highly unstable.

Problem 2: Vision Transformer (ViT)

I. Abstract and Motivation

This report evaluates the performance of a Vision Transformer(ViT) architecture on the MNIST handwritten digit classification task. The motivation is to explore how self attention

mechanisms perform on small scale image datasets when trained entirely from scratch, and to compare these results against traditional Convolutional Neural Network. By adapting a simplified ViT model to process 28×28 grayscale images, this section analyze its accuracy progression, parameter count, and overall sample efficiency.

II. Approach and Procedures

II.1 ViT Architecture

1. **Patch Embedding:** Unlike traditional CNN that process an image pixel by pixel, the Vision Transformer treats an image like a sentence. Patch embedding is the process of dividing the input image into a grid of fixed size, non-overlapping squares(patches). These 2D patches are then flattened into 1D vectors and passed through a trainable linear projection layer to map them into a constant, lower dimensional space. Transformers were originally designed for Natural Language Processing to process sequences of word tokens. Patch embedding bridges the gap between vision and language by converting a 2D continuous image into a discrete sequence of 1D visual tokens that the standard Transformer encoder can digest.
2. **Positional Encoding:** Because the Transformer architecture processes all tokens simultaneously rather than sequentially, it has no sense of order or geometry. To solve this, a positional encoding is added directly to each embedded patch token before it enters the Transformer blocks. The primary function of positional encoding is to inject spatial awareness into the model. It acts as a set of coordinates, allowing the Transformer to retain information about where each specific patch lived in the 2D layout of the image. Without it, the model could not understand the patches' relative positions.
3. **Multi-Head Attention:** This is the core engine inside the Transformer encoder blocks. Self-attention allows each patch to look at every other patch in the sequence to determine context. Multi-head means this self-attention process is executed multiple times in parallel within the same layer. The results from all the independent heads are concatenated and linearly transformed to produce the final output. Multi-head attention allows the model to capture complex spatial dependencies and relationships across the entire image. By using multiple heads, the network can simultaneously attend to different types of information giving the model a holistic understanding of the visual scene.(For example, one head might focus on local textures and edges, while another focuses on global structures.)

II.2 Compare classification performance over the MNIST dataset

The model was configured specifically for the MNIST dataset: 28×28 single channel images were divided into uniform patches, flattened, and linearly embedded into a sequence before being fed into a standard Transformer encoder along with positional embeddings. The network was compiled with a cross entropy loss function and trained using the Adam

optimizer (learning rate = 0.001). I tracked both training and test accuracies at the end of each epoch to monitor convergence and overfitting over a total of 20 epochs.

III. Experimental Results

III.1 Compare classification performance over the MNIST dataset

The model achieved a final test accuracy of 98.23% after 20 epochs. As illustrated in Figure 8, the model converged rapidly within the first 5 epochs and then entered a phase of gradual refinement.

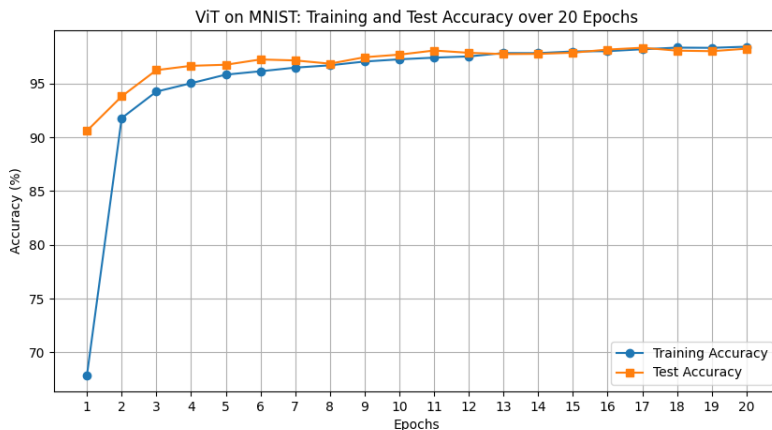


Figure 8: Training and Test Accuracy curves of the ViT over 20 epochs.

Epoch	Train Accuracy (%)	Test Accuracy (%)
1	67.91	90.60
5	95.82	96.74
10	97.24	97.68
15	97.97	97.86
20	98.42	98.23

Table 2: Summary of Accuracy Progress

Based on the programmatic evaluation of the PyTorch model tensors, the simplified ViT architecture contains 210,058 trainable parameters. Upon saving the model’s state dictionary to disk, the physical file size was calculated to be 0.84 MB. This confirms that the model is relatively lightweight, balancing capacity with computational efficiency.

IV. Discussion

IV..1 Compare classification performance over the MNIST dataset

When comparing the ViT’s performance against CNN, I observe that the CNN generally outperforms the ViT on the MNIST dataset. A CNN effortlessly achieves $> 99\%$ test accuracy on MNIST within fewer epochs, whereas our ViT plateaued at 98.23%.

- **Inductive Biases:** CNNs possess built-in inductive biases. The convolution operation assumes that adjacent pixels are highly correlated. ViTs treat the image as a flat sequence of patches and must learn these 2D spatial relationships purely from data via global self-attention.
- **# of Data:** Because ViTs start with fewer assumptions about the data structure, they require significantly more data to achieve the same performance as a CNN. The 60,000 images in the MNIST dataset are enough to learn a good representation, but they are not enough compared to the much larger datasets where ViT outperform CNN.
- **Patch Artifacts:** MNIST digits consist of continuous, smooth strokes. Slicing these small 28×28 images into discrete patches can disrupt the continuous flow of the digit’s edges, making the task harder for the attention mechanism without the smoothing effect of overlapping convolutions.